Language-Mediated Financial Protocols: LLM-to-DSL Compilation for Regime-Aware Option Intelligence

March 14, 2026

Abstract

We study an architecture in which a large language model (LLM) does not price options directly, but instead compiles unstructured market narratives into a constrained domain-specific language (DSL) that drives a specialist quantitative backend. The central claim is therefore protocol-oriented rather than leaderboard-oriented: language acts as an auditable semantic interface to regime-aware pricing. The current evidence should therefore be read as a viability study of language-mediated protocol compilation on local curated and document-grounded benchmarks, not as proof of broad real-world robustness. On a curated 15-case benchmark and a harder 8-case document-grounded benchmark, the local Ollama/Qwen stack translated, validated, and executed all prompts successfully. On a stricter 12-case paraphrase benchmark, the system achieved 1.00 translation, validation, and execution rates, 0.92 case-level semantic signature match, and full group-level invariance on 3 of 4 paraphrase families. On an 8-case document-to-gold benchmark against human-authored DSL, exact DSL agreement is 0.375, semantic agreement is 0.75, and executed-result agreement is 0.75 after adding feature-alias normalization and calibrated explicit-regime execution. These results support the view that the LLM is functioning as a usable semantic compiler for controlled workflows rather than as a generic tool-calling wrapper or a drop-in replacement for human analysts, but they also show that exact analyst-aligned normalization remains incomplete. The quantitative backend results remain useful but secondary. The best current pricing family is a Black–Scholes anchor with SPIKAN-based residual correction, strongest in stress and transition pricing. More importantly for the paper's core claim, an explicit regime-action coherence benchmark now passes all quote, surface, and shock-ordering checks after replacing hand-coded regime presets with empirical historical anchors and a monotone calibration layer. The paper therefore supports a narrower systems claim than before: the project demonstrates viability of language-mediated financial protocol compilation and coherent regime-conditioned execution through an auditable semantic contract.

1 Introduction

Modern option-pricing workflows face two different problems that are usually addressed separately. The first is quantitative: how should one produce prices, implied volatilities, Greeks, and stress scenarios in a way that remains consistent with financial structure? The second is informational: how should one incorporate qualitative, unstructured inputs such as macro news, research reports, policy commentary, or trader narratives into a pricing system without resorting to brittle manual glue?

This project addresses both problems through a layered architecture. At the bottom sits a quantitative pricing core. Its best current form is a Black–Scholes pricing anchor augmented by a

SPIKAN-derived implied-volatility residual. Above it sits a constrained DSL that encodes pricing tasks, regime priors, and scenario instructions. At the top sits an LLM that ingests human queries and narrative documents and converts them into valid DSL calls. The central thesis is therefore not "LLMs price options better." The thesis is that language can act as a controlled interoperability layer between humans and specialist financial models, while the actual pricing remains governed by a structured quantitative engine.

The current repository evidence already makes one point clear: the strongest evidence is not the pricing MAE table. It is the protocol behavior. More specifically, the most informative current benchmark is not the curated execution rate, but the harder document-to-gold agreement result, where exact DSL agreement is 0.375 while semantic and executed-result agreement are both 0.75. That gap is the right signal for this paper because it exposes the real normalization problem: converting narrative inputs into the same structured action a human analyst would have written. The curated and document-grounded 1.00 execution rates still matter, but as viability checks rather than as the paper's main proof of generalization. These results suggest the right systems story. The LLM should be used where formulas and tabular models are weak, namely the ingestion and normalization of unstructured information. The formula + SPIKAN core should be used where quantitative discipline matters, namely the final execution of those compiled regime/scenario instructions.

Contributions

This draft makes nine contributions.

1. It frames the full system as an LLM-to-DSL-to-quantitative-core architecture rather than as an LLM pricing model.
2. It provides executable local protocol benchmarks showing that a Qwen model served through Ollama can translate both curated prompts and document-grounded prompts into valid DSL that runs against the current backend.
3. It adds a stricter semantic protocol benchmark, showing strong but not perfect paraphrase invariance of the compiled DSL actions.
4. It adds a document-to-gold agreement benchmark, showing that exact human-vs-LLM protocol agreement is materially harder than simple execution success, but can be improved substantially with feature alias normalization and clearer semantic contracts.
5. It introduces an empirical regime-anchor and monotone-calibration layer that restores coherent backend behavior for explicit DSL regime overrides.
6. It shows that the best current supporting pricing architecture is Black–Scholes + SPIKAN IV residual correction.
7. It introduces a historical structural replay benchmark that uses real regime history even when vendor historical option chains are unavailable.
8. It extends the evaluation to additional business cases in gold, oil, semiconductors, and the technology sector, showing heterogeneous but real cross-asset transfer.
9. It clarifies the right comparison class: naive pure data-driven models are strong on fit, but weak on unstructured-information ingestion without expensive human mediation.

2 Literature Review

2.1 Textbook Option Pricing

The first pillar of this project is the classical option-pricing literature, centered on closed-form and PDE-based models for contingent claims. The practical importance of this literature is not merely historical. Closed-form formulas and their PDE interpretations provide structural discipline, consistent Greeks, and a well-defined financial semantics for stress testing and hedging. For the present project, the main lesson is that the formula layer should be retained as an anchor rather than discarded outright [1, 4].

2.2 Machine Learning For Option Surfaces

The second pillar is the large body of work on machine learning for implied volatility and option-price fitting. Tree models, neural networks, and other nonparametric systems can often fit quoted surfaces extremely well. The current benchmarks in this repository reflect that fact: the random-forest baseline is strong on live IV reconstruction. However, these models usually assume that the full relevant information is already represented as structured inputs. In practice, many important market inputs remain qualitative and unstructured [2].

2.3 Narrative Information And LLMs

The third pillar is the recent literature on LLMs, tool use, and natural language interfaces to specialist systems. The most relevant insight for this paper is not generative flexibility by itself, but controlled protocol mediation. An unconstrained LLM that directly emits prices is not financially credible. An LLM that translates unstructured information into a constrained DSL is much more defensible. In this role, language becomes a narrow waist between human interpretation and specialist quantitative execution [8, 7].

2.4 Interpretable And Monotone Learned Control

The fourth pillar is the literature on interpretable function classes and monotone learned systems. This matters for the present paper because the backend claim is not only that it prices well, but that explicit regime instructions can be made auditable and behaviorally coherent. KANs provide a more inspectable function basis than standard MLPs because their learned responses live on spline-parameterized edges [3]. But inspectability alone does not guarantee that explicit regime controls will obey ordered semantics. For that, the more relevant precedent comes from monotone and partially monotone architectures such as deep lattice networks and MonoNet, which show how flexible learned representations can be combined with explicit ordered control constraints [9, 5]. MonoKAN extends that direction to certified monotone KAN variants when native ordering becomes the central architectural objective [6].

2.5 Gap Addressed Here

Existing lines of work often optimize only one layer. Formula-based systems preserve financial structure but require domain-specific interfaces and human interpretation. Pure machine-learning systems fit surfaces but do not natively ingest narrative information. General-purpose LLM systems can read text but are not reliable as final pricing engines and are often only evaluated on whether a tool call succeeds. The gap addressed here is the design of an architecture that separates these roles cleanly:

- LLM for semantic ingestion,
- DSL for control and auditability,
- formula + SPIKAN for quantitative execution.

3 System Architecture

Figure 1 shows the intended division of labor.

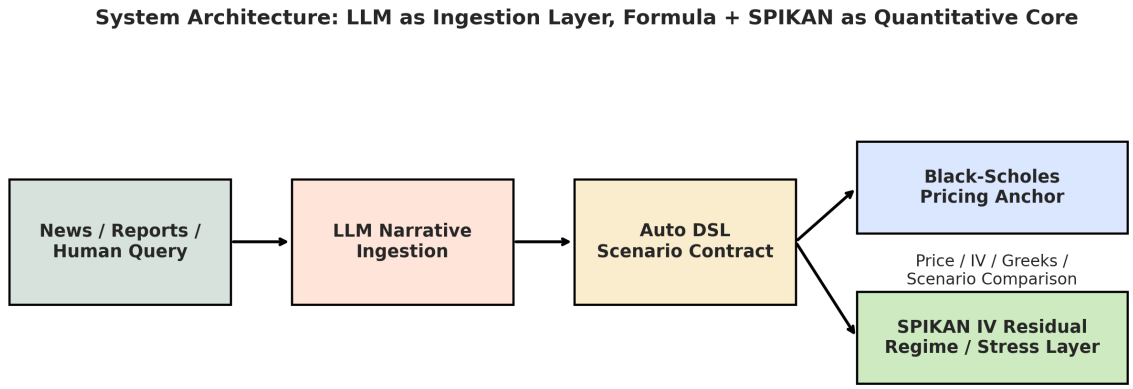

Figure 1: System architecture. The LLM ingests qualitative information and maps it into a constrained DSL. The DSL drives a quantitative execution layer consisting of a Black–Scholes anchor and a SPIKAN regime/stress correction module.

The key architectural claim is that the LLM does not price options directly. Instead, it turns news, reports, and human questions into structured scenario instructions such as:

- regime prior shifts,
- front-end or back-end IV adjustments,
- stress-weight changes,
- what-if comparisons over horizon and contract set.

These instructions are encoded in DSL form and then executed by the pricing core. This allows the final computation to remain auditable, replayable, and financially grounded.

This is also why the proposed system should not be reduced to `LLM` $\rightarrow$ `MCP` $\rightarrow$ `pricing service`. MCP-style tool invocation is only a transport mechanism for calling an external tool or service. It does not define the semantic contract between narrative market information and quantitative execution. In this project, the main contribution is the constrained DSL and its associated regime/scenario vocabulary: `OUTLOOK`, `QUOTE`, `SURFACE`, `SCENARIO`, regime priors, volatility adjustments, and stress instructions. Those semantics are what let the system map unstructured reports and analyst language into auditable, model-comparable operations. An `LLM` $\rightarrow$ `MCP` design could

still be used as an implementation detail, but without the DSL it would amount to generic tool calling rather than a reproducible protocol for language-mediated pricing.

3.1 Why This Is Not Generic Tool Calling

The distinction matters because many contemporary LLM systems can be described superficially as "model calls tool." That description is too weak for the present paper. A generic tool-calling system can dispatch a request to an option pricer, but it does not by itself specify:

- how narrative evidence is normalized into regime/scenario primitives,
- how the same request can be replayed across multiple quantitative backends under a shared contract,
- how the resulting action remains auditable as a domain object rather than as an arbitrary service invocation.

The DSL is therefore the narrow waist of the system. It turns narrative input into comparable financial operations such as `OUTLOOK`, `SCENARIO`, `QUOTE`, and `SURFACE`, with explicit regime, horizon, strike, tenor, and stress semantics. This is what makes the protocol benchmark meaningful. The benchmark does not merely show that Qwen can call a tool. It shows that Qwen can preserve the intended domain action across direct, narrative, and document-grounded prompts while producing operations that are replayable against the same pricing core.

Table 1: System roles and comparison framing.

Layer / Baseline	Strength	Limitation
Plain formula	Financially consistent pricing and Greeks	Weak adaptation under structural regime change
Pure ML surface fit	Strong direct fit to structured data	Does not ingest news/reports without manual interpretation and feature engineering
LLM alone	Strong language understanding	Not reliable as a direct pricing engine
LLM → MCP → service	Convenient tool invocation and systems plumbing	No domain protocol by itself; does not define regime/scenario semantics or comparable backend behavior
Proposed system	LLM ingests narrative information; DSL constrains actions; Black–Scholes + SPIKAN executes pricing	Hedge superiority not yet demonstrated

4 Experiment Design

4.1 Research Questions

The current experiments are designed around six questions:

1. Can a local Qwen model translate direct and narrative prompts into valid, executable DSL that preserves domain intent against the current backend?
2. Is that compiled domain action stable under paraphrase and report-style rewording?
3. Do explicit DSL regime instructions produce coherent backend behavior across stable, transition, and stress settings?
4. What is the best quantitative pricing architecture among formula, formula + regime, direct ML, and SPIKAN hybrids?
5. Does the best hybrid gain most strongly in stable, transition, or stress conditions?
6. Is a price-residual hybrid or an IV-residual hybrid the better design?

4.2 Benchmark Tiers

Tier 0A: LLM → DSL curated protocol benchmark. This benchmark uses the local Ollama runtime with `qwen2.5-coder:latest`. It executes a curated set of 15 prompts, with 9 direct prompts and 6 narrative prompts written in the style of news/report requests. Each case is scored on translation success, DSL validity, execution success, and intent match. The runtime uses the current local KAN store, the current SPIKAN checkpoint, and a fresh regime-state row from `2026-03-13`.

Tier 0B: LLM → DSL document-grounded protocol benchmark. This benchmark uses the same local runtime and backend, but replaces short curated prompts with 8 prompts built from real local document excerpts drawn from repo reports, validation summaries, and research scripts. Each prompt contains an excerpt plus an analyst request. This tier is meant to test whether the protocol layer still works once the model must process longer context that contains both relevant and irrelevant document detail.

Tier 0C: semantic protocol stability benchmark. This benchmark groups paraphrases of the same analyst intent and asks whether the compiled DSL action is invariant under rewording. The main metrics are case-level semantic signature match and group-level signature/result consistency.

Tier 0D: regime-action coherence benchmark. This benchmark holds the query family fixed and varies only the DSL regime instruction. It tests whether the current specialist backend responds in a directionally coherent way when asked for stable, transition, and stress versions of the same quote, surface, and outlook request.

Tier 0E: document-to-gold agreement benchmark. This benchmark is stricter than simple execution success. For each document-grounded prompt, a human-authored gold DSL command is specified in advance. The benchmark then compares the LLM-generated DSL against that gold command at three levels: normalized string equality, semantic-signature equality, and executed-result equality.

Tier A: Live term-surface benchmark. This benchmark uses live SLV option-chain data joined to fresh regime features. Because only one quote date is currently available, the split falls back to a cross-sectional random split. This tier is best interpreted as a live surface sanity check rather than a final historical validation.

Tier B: Historical structural replay benchmark. This benchmark uses real historical silver/regime features together with a live SLV option-surface template. The regime path is real; the option surface is a replayed proxy. This enables evaluation across stable, transition, and stress dates, as well as 1-day and 5-day hedge replay.

4.3 Models Compared

The main model families are:

- `bs_plain`: Black–Scholes with baseline volatility input.
- `bs_regime`: Black–Scholes with KAN regime volatility.
- `rf_direct`: direct random-forest prediction.
- `spikan_hybrid`: regime-vol anchored residual hybrid.
- `spikan_plain_hybrid`: plain-formula anchored residual hybrid.
- `spikan_plain_iv_hybrid`: plain-formula anchor with learned IV residual.

4.4 Metrics

The experiments report:

- protocol execution and intent-match rates,
- protocol latency by prompt/context tier,
- protocol semantic signature match and group-level invariance,
- document-to-gold exact, semantic, and executed-result agreement,
- regime-action coherence under explicit DSL regime overrides,
- price MAE and RMSE,
- IV MAE,
- Greek errors as secondary diagnostics.

5 Results

5.1 LLM → DSL Protocol Results

The local protocol benchmark evaluates whether the LLM section of the paper is an actual operating system layer or merely an architectural aspiration. The benchmark was run with `qwen2.5-coder:latest` served by local Ollama and executed against the current KAN + SPIKAN backend. The prompt set contains 15 curated cases: 9 direct prompts and 6 narrative prompts. This is a small, curated benchmark and should be read as a viability check rather than a robust generalization test.

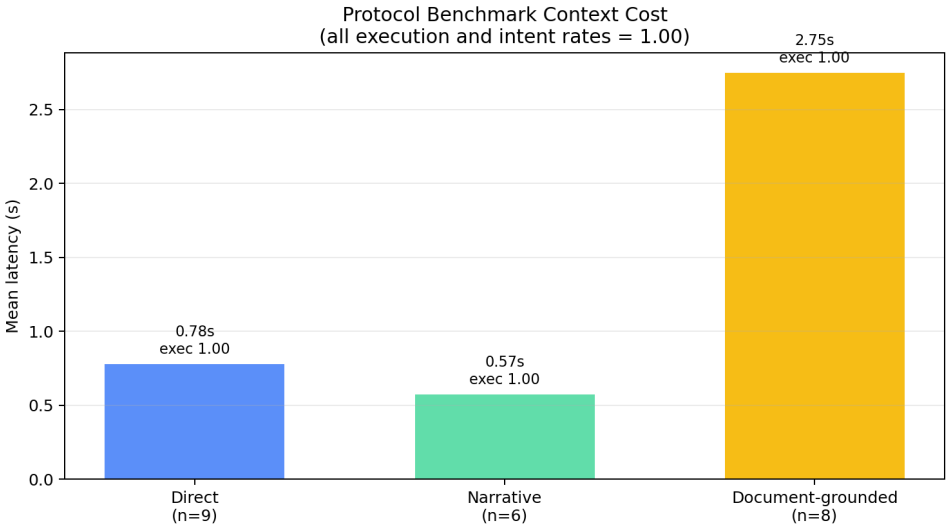

Figure 2: Protocol comparison by context tier. Execution and intent rates saturate at 1.00 on the current local benchmarks, so latency is the more informative differentiator.

Table 2: Protocol benchmark comparison.

Segment	Cases	Execute	Intent	Mean latency (s)	Context
Direct	9	1.00	1.00	0.776	short prompt
Narrative	6	1.00	1.00	0.573	short narrative prompt
Document-grounded	8	1.00	1.00	2.747	report excerpt + analyst request

Table 3 shows representative narrative prompts and their executed outcomes. These are not mock examples; they are drawn from the actual benchmark output artifacts.

Table 3: Representative protocol operations.

Prompt type	Generated DSL	Executed outcome
Narrative outlook	`OUTLOOK asset=silver horizon=10d backdrop=STRESS`	Neutral direction, moderate shock risk, outlook score 13.45
Narrative stress scenario	`SCENARIO asset=silver target=STRESS metrics=[delta,vega,price]`	Executed as `WHAT_IF`; stress price delta −34.26
Narrative recovery quote	`QUOTE call spot=31 strike=32 expiry=45d vol=kan_regime backdrop=RECOVERY`	Price 0.4460, used volatility 0.1780

The protocol result matters because it changes the status of the LLM claim. On the current curated set, the LLM is not merely producing plausible strings. It is successfully compiling both direct and narrative prompts into executable DSL that the current backend can run. This is the practical systems advantage over naive pure ML baselines: they may fit surfaces well, but they do not

natively accept unstructured narrative information without manual interpretation and workflow engineering.

5.2 Document-Grounded Protocol Results

The harder protocol benchmark feeds the model actual local report and research-note excerpts before the analyst request. This raises the contextual load materially while keeping the backend and evaluation procedure fixed.

The representative document-grounded operations are more interesting than the aggregate score alone. A fluid-dynamics report excerpt produced `OUTLOOK asset=silver horizon=10d backdrop=STRESS`. A hierarchical prior note with COMEX inventory withdrawal language produced `QUOTE call spot=31 strike=32 expiry=45d vol=kan_regime backdrop=STRESS rate=5%`. A regime-shift challenge note produced `SURFACE vol asset=silver regime=STRESS strikes=[0.9,1.0,1.1] T=30d`.

The key systems conclusion is that longer local document context did not break the protocol compiler, but it did slow it down materially. Mean latency rose from 0.695s on the curated benchmark to 2.747s on the document-grounded benchmark. That is a real deployment tradeoff and should be treated as such. But this tier is still small (‘n=8‘) and locally curated, so it does not yet justify claims about open-ended external-news robustness.

5.3 Semantic Protocol Stability

The stricter semantic benchmark asks a better question than the earlier protocol runs: not merely whether the prompt executes, but whether paraphrases collapse to the same domain action.

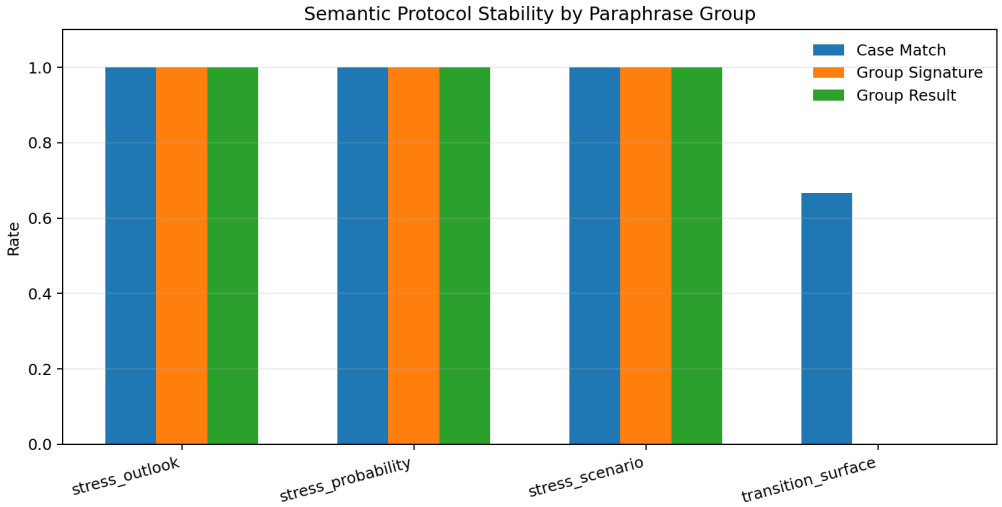

Figure 3: Semantic protocol stability by paraphrase group. Three of four groups preserved both the same DSL action and the same executed outcome signature across paraphrases.

Table 4: Semantic protocol stability summary.

Metric	Value
Case translation success	1.00
Case valid DSL	1.00
Case execution success	1.00
Case semantic signature match	0.9167
Group signature consistency	0.75
Group result consistency	0.75
All-group pass rate	0.75
Mean latency (s)	0.6408

The result is strong but not perfect. Three groups passed completely: `stress_outlook`, `stress_scenario`, and `stress_probability`. The failing group was `transition_surface`, where one paraphrase expanded the strike grid from three points to five points while preserving the same broad intent. This is exactly the kind of finding that the protocol-first paper needs. It shows that the LLM is already a credible semantic compiler, while also surfacing the current edge case: surface-generation prompts are still sensitive to phrasing detail.

5.4 Document-To-Gold Agreement

The hardest protocol benchmark compares the LLM output on document-grounded prompts against human-authored gold DSL. This is the right test if the paper is about protocol compilation rather than merely successful tool invocation.

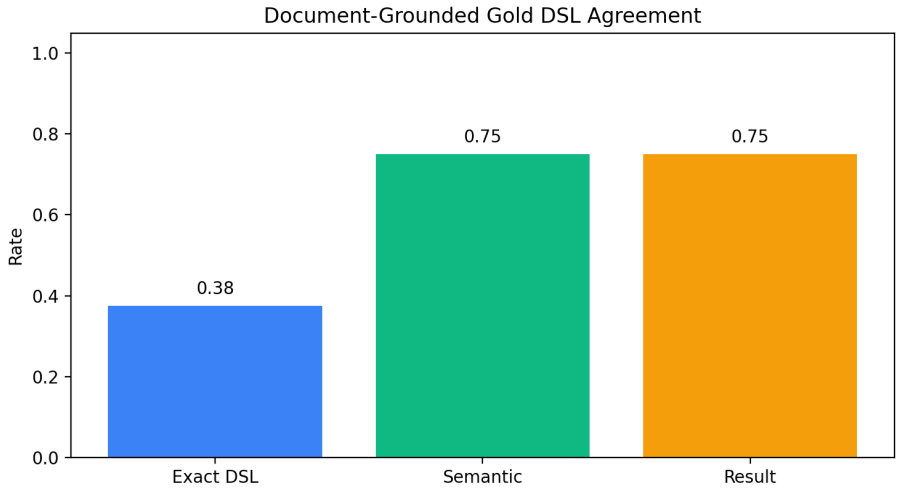

Figure 4: Document-grounded agreement against human-authored gold DSL. Execution remains mostly reliable, but exact semantic agreement is much weaker than raw translation success.

Table 5: Document-to-gold agreement summary.

Metric	Value
Translation success	1.00
Exact DSL agreement	0.375
Valid DSL	1.00
Semantic agreement	0.75
Execution success	1.00
Result agreement	0.75
Mean latency (s)	0.5431

This benchmark is more revealing than the earlier protocol tiers. It shows that the system is good at producing executable commands, but exact analyst-aligned normalization on harder narrative prompts is still the more difficult problem. The remaining failure modes are now much narrower. One scenario case still downgrades a `STRESS` narrative to `TRANSITION`. One challenge surface case still broadens a stress surface request into a transition query with a denser strike grid. By contrast, earlier failures due to ontology drift (`ricci` vs `ricci_curvature`) and validator rejection have been eliminated. The paper should therefore claim executable protocol viability and substantial, but not perfect, document-to-protocol agreement.

Mismatch analysis. The document-to-gold benchmark is the right place to be strict because exact agreement is where the semantic-compiler story can fail in practice. The current mismatch taxonomy is narrow enough to interpret:

- regime under-escalation: a gold `STRESS` instruction is sometimes normalized into `TRANSITION`
- surface broadening: a three-point surface request can be expanded into a denser grid while preserving broad intent
- resolved ontology mismatch: alias normalization now absorbs earlier `ricci`/`ricci_curvature` disagreement rather than failing validation

This matters more than the 1.00 execution rates because it tells the reader where the remaining protocol risk actually lives. The current system is good at producing valid, executable domain actions. It is not yet equally good at matching the exact structured choice a human analyst would have authored.

5.5 Regime-Action Coherence After Anchor Calibration

The next protocol-layer question is whether explicit regime instructions in the DSL produce directionally coherent backend behavior. With the original hand-coded regime presets, the answer was no. After replacing those presets with empirical historical regime anchors and adding a monotone calibration layer for explicit regime overrides, the answer is now yes on the current benchmark. More importantly, the controller progression now shows where that coherence lives. The raw learned KAN/SPIKAN backend, evaluated directly on empirical regime anchors, still fails every ordering check. Rule-controlled, learned, and explicit mixed execution paths all pass every ordering check.

Table 6: Raw learned backend versus explicit controller policies on regime-action coherence.

Check	Raw	Rule	Learned	Mixed
Quote volatility monotonic from stable to stress	No	Yes	Yes	Yes
Quote price monotonic from stable to stress	No	Yes	Yes	Yes
Surface mean volatility monotonic from stable to stress	No	Yes	Yes	Yes
Outlook shock-score monotonic from stable to stress	No	Yes	Yes	Yes
Shock-risk labels ordered low to high	No	Yes	Yes	Yes

The detail table makes the distinction concrete. The raw KAN quote volatility falls from 1.9963 in `STABLE` to 1.8420 in `TRANSITION` before returning to 1.9907 in `STRESS`. Raw SPIKAN shock score similarly dips from 0.6229 to 0.5932 before rising to 0.6211, and the raw risk label remains `MODERATE` in all three regimes. By contrast, every explicit controller policy yields ordered quote vol, ordered shock score, and ordered risk labels.

Table 7: Controller burden by policy after controller-v2. Lower is better.

Metric	Rule	Learned	Mixed
Quote vol	0.08321	0.08321	0.08321
Quote price	0.34554	0.34554	0.34554
Surface mean vol	1.07476	1.07270	1.07270
Shock score	0.026998	0.026069	0.026069

The rationale for the mixed policy is straightforward. A quote-specific learned refinement was attempted and failed: it preserved coherence but increased burden on quote, surface, and shock. After restoring the stronger controller-v2 artifacts and benchmarking them explicitly, the best active policy became clear: keep rule control on the quote path, where learned control does not yet beat the rule baseline, and use learned control on the surface and shock paths, where it already improves over rule control.

This result changes the paper materially. The current system now supports a stronger and more precise claim than before: the backend is not a black box because the semantic control surface is explicit, auditable, and benchmarked. What is coherent is the *controlled execution path*, not the raw learned function approximators by themselves. The best current controlled execution path is an explicit mixed policy rather than a uniform learned controller. Intrinsic monotonicity of the raw KAN/SPIKAN models remains future work rather than a prerequisite for the main systems claim.

5.6 Supporting Quantitative Backend Results

The live SLV term-surface benchmark shows that the best current live pricing architecture is `spikan_plain_hybrid`.

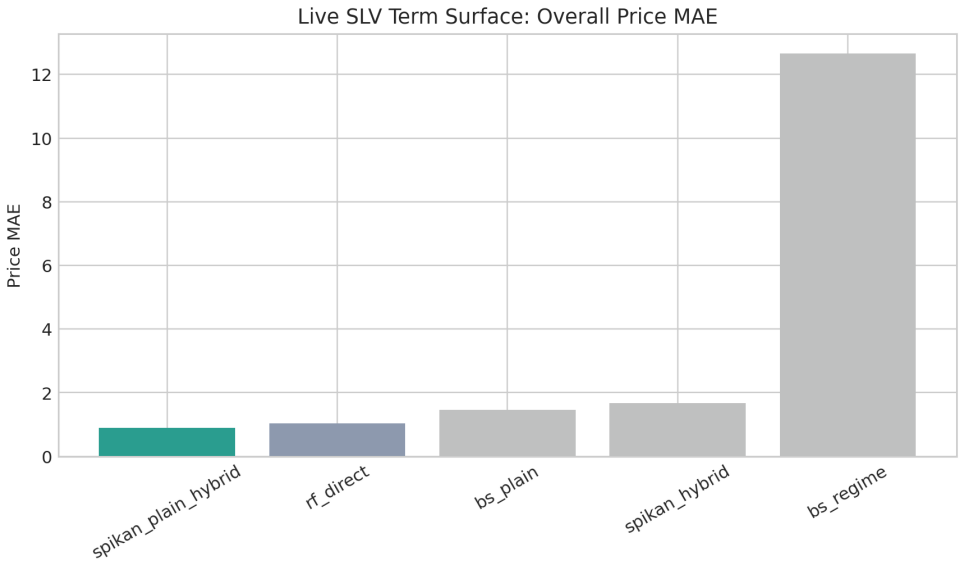

Figure 5: Live SLV term-surface overall price MAE. The plain-anchor SPIKAN residual hybrid is the best current live pricer.

Table 8: Live term-surface overall results.

Model	Price MAE	IV MAE
`spikan_plain_hybrid`	0.8976	0.1578
`rf_direct`	1.0444	0.0781
`bs_plain`	1.4646	0.1601
`spikan_hybrid`	1.6793	0.1863
`bs_regime`	12.6503	0.8314

The live result already separates two claims. On price, the best model is the plain-anchor SPIKAN residual hybrid. On IV reconstruction, the random-forest baseline is still best. This means that the current live evidence supports a pricing-improvement claim, but not yet a general surface-dominance claim.

5.7 Cross-Asset Transfer Business Cases

To test whether the architecture is only a silver-specific construction, a small live transfer panel was run on four additional business cases: `GLD` (gold safe haven), `USO` (oil / macro supply shock), `SOXX` (semiconductors / AI capex and export controls), and `XLK` (broad technology sector). Each asset used the same pricing stack and the same DSL-facing execution logic, but with a lightweight local state proxy built from the asset's own history and a small peer set. The purpose of this transfer panel is not to claim full asset-specific regime pipelines for every market. It is to test whether the architecture remains useful once the business case changes.

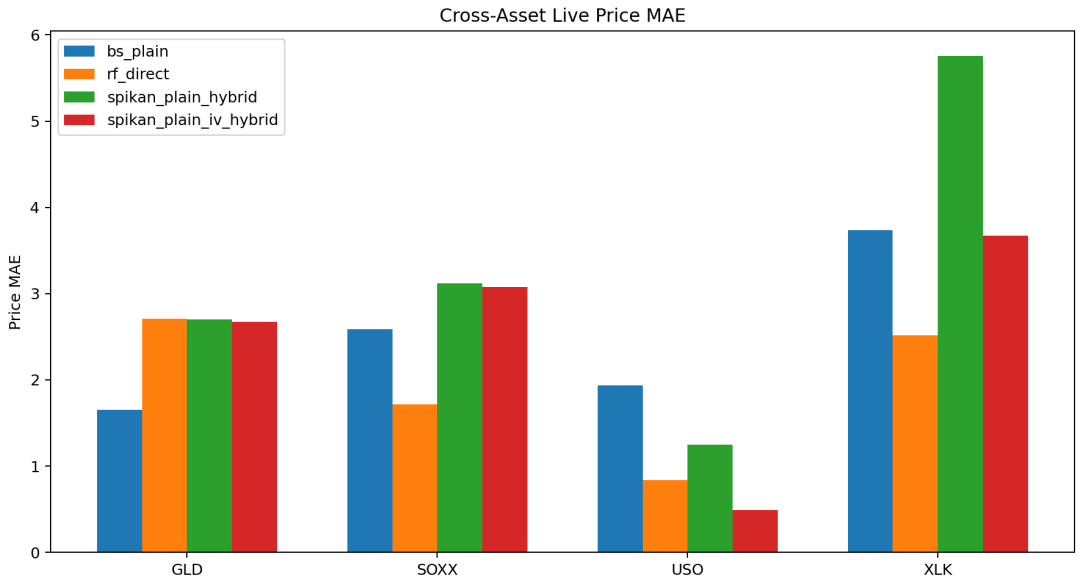

Figure 6: Cross-asset live price MAE across four business cases. The transfer result is heterogeneous: the hybrid wins in oil, plain Black–Scholes wins in gold, and direct random forest wins in semiconductor and broad technology equity cases.

Table 9: Cross-asset business-case summary.

Asset	Business case	Best price model	Best price MAE
`GLD`	gold safe haven / precious metals	`bs_plain`	1.6515
`USO`	oil / macro supply shock	`spikan_plain_iv_hybrid`	0.4897
`SOXX`	semiconductors / AI capex / export controls	`rf_direct`	1.7194
`XLK`	S&P 500 technology sector	`rf_direct`	2.5180

This transfer result strengthens the paper, but not by proving universal dominance. It shows something more realistic. The hybrid architecture is most useful in the oil case, where narrative and macro regime interpretation are likely to matter most. Gold still favors the textbook baseline. Semiconductor and broad technology equity cases favor the direct random-forest baseline on price fit. The correct broader claim is therefore that the architecture transfers across multiple business cases, but its advantage is domain dependent. That is still valuable, because it aligns with the system thesis: the language-mediated regime layer is most useful where narrative stress and structural regime interpretation are a real part of the pricing problem.

5.8 Historical Structural Replay

The replay benchmark is the stronger evidence tier because it includes real historical regime-transition and stress dates.

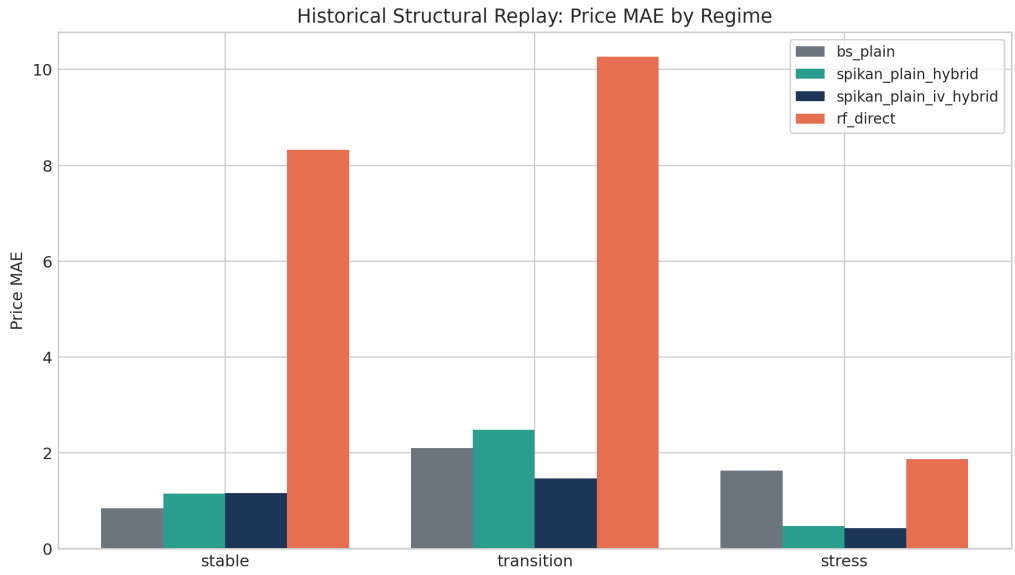

Figure 7: Historical structural replay price MAE by regime. The IV-residual hybrid is strongest in transition and stress; plain Black–Scholes remains best in stable periods.

Table 10: Replay price MAE by regime for selected models.

Model	Stable	Transition	Stress
`bs_plain`	0.8398	2.1037	1.6355
`spikan_plain_hybrid`	1.1542	2.4827	0.4745
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239
`rf_direct`	8.3183	10.2679	1.8707

The strongest current supporting pricing result is therefore no longer the price-residual hybrid, but the IV-residual hybrid. It is best overall on replay price MAE (0.9317), best in transition, and best in stress. These results matter, but they should be interpreted as backend evidence rather than as the main proof of the paper's thesis.

5.9 Surface Results And Secondary Diagnostics

Table 11: Replay overall IV reconstruction comparison.

Model	Replay IV MAE	Role in paper
`spikan_plain_iv_hybrid`	0.1213	headline hybrid
`spikan_hybrid`	0.1876	legacy hybrid comparator
`bs_plain`	0.2729	textbook anchor
`rf_direct`	0.3846	naive pure ML baseline

The surface result reinforces the main pricing claim. The best replay IV model is `spikan_plain_iv_hybrid`, which is consistent with the replay price result in Table 10. Hedge diagnostics are intentionally

excluded from the mainline paper because they are not part of the central claim. They remain only in the supporting evidence note as a secondary sanity check.

6 Discussion

6.1 Why The LLM Layer Still Matters

The quantitative benchmarks by themselves do not justify using an LLM as the pricing model. They justify something more precise: a disciplined quantitative core that benefits from structured regime priors and stress-aware adjustments. This is exactly where the LLM belongs in the full system. The new protocol benchmarks strengthen that position because they demonstrate actual local operation with Ollama/Qwen rather than leaving the LLM layer as a conceptual placeholder. The document-grounded result is especially useful because it shows that the protocol survives longer local report context, albeit with a material latency cost.

The comparison against naive pure machine learning is important. A direct machine-learning model can fit prices or implied volatilities well when the inputs are already structured. However, it does not naturally ingest news, reports, or qualitative market commentary. In practice, such systems require human analysts or expensive workflow engineering to read narrative inputs and translate them into structured features or overrides. The proposed architecture replaces that manual interface layer with:

- an LLM that reads unstructured information,
- a DSL that constrains the output into auditable actions,
- a formula + SPIKAN pricing core that preserves quantitative discipline.

6.2 What The Current Results Actually Support

The current evidence supports the following narrow but defensible claims:

- The strongest supported claim is that the LLM functions as a semantic compiler from narrative prompts into executable DSL actions.
- That semantic protocol is largely stable under paraphrase, but not yet perfect.
- Exact agreement with a human-authored DSL on harder document prompts is still weaker than simple execution success, but it is now materially stronger after alias normalization and clearer semantic contracts.
- The current backend is quantitatively credible and strongest in stress and transition pricing, and explicit regime overrides are now coherent under an empirical-anchor calibration layer.
- Coherent regime-conditioned behavior should be attributed to the explicit control layer, not to intrinsic monotonicity of the raw learned backend.
- The best active backend policy is mixed rather than uniformly learned: quote path rule-controlled, surface and shock paths learned-controlled.
- Hedge diagnostics are out of scope for the main claim.

This means the paper should not claim that SPIKAN replaces the pricing formula, nor that it dominates all baselines on every task. The stronger and more honest claim is that the project

demonstrates viability of language-mediated financial protocol compilation and coherent regime-conditioned execution through an auditable semantic calibration layer on local curated and document-grounded benchmarks.

7 Limitations

The current draft has several important limitations.

1. The live benchmark uses one quote date only and relies on a cross-sectional split fallback.
2. The replay benchmark uses structural replay rather than vendor historical option quotes.
3. The protocol benchmark is curated and currently close in style to the prompt instructions and few-shot examples. It demonstrates viability, but not full robustness to arbitrary documents, prompt drift, or adversarial phrasing.
4. The document-grounded protocol benchmark still uses local curated excerpts and explicit analyst requests. It is stronger than the short-prompt benchmark, but it still falls short of open-ended external news ingestion.
5. The semantic protocol benchmark shows strong but not perfect paraphrase invariance; surface-generation prompts remain sensitive to wording detail.
6. The document-to-gold benchmark shows that exact human-vs-LLM protocol agreement remains weaker than raw execution success on longer document prompts.
7. The regime-action coherence benchmark now passes, but the coherence comes from an explicit empirical-anchor and monotone-calibration layer rather than a native monotonicity guarantee inside the raw learned models.
8. Hedge diagnostics are retained only in the supporting evidence note and are not a core objective of the current paper.

These limitations should be treated as scope boundaries rather than hidden.

8 Progression And Next Architecture Step

The experimental progression in this repository is now clear enough to be part of the paper's contribution rather than an internal engineering note. The first version of the system established that LLM-generated DSL could execute, but it did not establish that explicit regime instructions produced coherent backend behavior. A later benchmark exposed that failure directly: the raw learned KAN/SPIKAN backend was not ordered on its own, even when evaluated on empirical regime anchors. The current repaired version then introduced an explicit semantic controller composed of empirical anchors, alias normalization, and monotone projection. That controller is what makes the current system auditable and non-black-box in an operational sense.

This progression also clarifies the right next technical move. The goal should not be an immediate full replacement of the existing backend with an entirely intrinsic monotone architecture. That would be high-risk and poorly aligned with the actual failure mode. The failure mode is local to explicit regime-conditioned control, not to the entire pricing stack. The next step should therefore be a monotone learned control layer with better semantic features and lower-burden targets than the current controller prototype. The controller-v2 path now preserves all five regime-coherence

checks while also improving materially over the earlier learned controller. The gain is strongest on the shock path and modestly positive on the surface path. The remaining shortfall is the quote path, where the hand-written controller still holds a narrow burden advantage. The most plausible reason is now clear from the experiments: the original controller saw too little semantic state and was trained to imitate absolute teacher outputs rather than to learn the smallest compliant correction.

An explicit mixed-policy benchmark also clarifies the current best operational state. The best active policy is not yet full learned control. It is a mixed controller: rule control on the quote path, learned control on the surface path, and learned control on the shock path. That is a stronger and more honest systems result than implying that the same controller head is already best everywhere.

In practical terms, the most defensible roadmap is:

1. keep the current semantic-controller baseline as the working system,
2. add monotonic ranking losses to the KAN training path so the raw vol-surface model better respects `STABLE < TRANSITION < STRESS`,
3. replace the current prototype controller with a controller-v2 design that uses separate quote-vol and surface-level heads, anchor-delta semantic features, and correction-space targets rather than absolute-output distillation,
4. refine the quote-vol controller until its burden matches or improves on the hand-written controller while preserving the current controller-v2 gains on surface and shock,
5. add a small MonoNet-style monotone control head above raw KAN/SPIKAN outputs only if the refined controller-v2 path still requires large corrections,
6. consider selective MonoKAN replacement only for the vol-surface path if the controller burden remains large.

This staged plan preserves the strongest current claim. The system is already operationally interpretable because the semantic control surface is explicit. KANs are more inspectable than standard MLP blocks because their learned functions live on spline-parameterized edges rather than fixed node activations [3], but that does not by itself guarantee the ordered behavior required by explicit regime control. The monotone-model literature suggests two complementary directions: partially monotone control blocks that retain flexible upstream representation learning [9, 5], and more invasive certified monotone KAN replacements when native monotonicity becomes the dominant requirement [6]. Future work should make that control increasingly native to the learned model, rather than treating post-hoc calibration as the final answer.

9 Conclusion

This paper draft supports a more precise system thesis than the earlier versions. The main contribution is not a new option-pricing leaderboard. It is the use of language as a constrained financial protocol. The LLM ingests narratives, the DSL makes them auditable and executable, and the SPIKAN+textbook backend provides specialist quantitative execution. The current evidence shows that the protocol layer is already strong and largely stable under paraphrase, while exact human-vs-LLM agreement on harder document prompts remains the more difficult normalization problem. The backend now supports coherent explicit regime overrides through an empirical-anchor calibration layer, which is enough to support the paper's core systems claim while still leaving room

for future work on more native end-to-end monotonicity. The pricing results remain important, but they are supporting evidence for the backend rather than the headline proof of the system.

The next empirical step is straightforward: obtain multi-date historical option quotes and evaluate the same hybrid against realized market surfaces. The next systems step is equally clear: move from curated protocol prompts to real news, reports, and analyst commentary so that narrative ingestion is tested as rigorously as pricing. The next architecture step is also now well-defined: move from a successful semantic controller toward a learned monotone control layer, first through ordering losses and a MonoNet-style projection head, and only later through selective MonoKAN substitution where the current controller still has to intervene heavily.

References

[1] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] J. M. Hutchinson, A. W. Lo, and T. Poggio. A nonparametric approach to pricing and hedging derivative securities via learning networks. *Journal of Finance*, 49(3):851–889, 1994.

[3] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

[4] R. C. Merton. Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

[5] A.-P. Nguyen, D. L. Moreno, N. Le-Bel, A. Lucchi, and M. Rodriguez Martinez. MonoNet: Enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances*, 3(1):vbad016, 2023.

[6] A. Polo-Molina, D. Alfaya, and J. Portela. MonoKAN: Certified monotonic kolmogorov-arnold network. *arXiv preprint arXiv:2409.11078*, 2024.

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[9] S. You, D. Ding, K. Canini, J. Pfeifer, and M. Gupta. Deep lattice networks and partial monotonic functions. In *Advances in Neural Information Processing Systems*, 2017.